

UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
DCC606 – ANÁLISE DE ALGORITMOS – 2026
PROF. DR. HEBERT OLIVEIRA ROCHA

Coloração de Vértices em Grafos para Otimização de Cronogramas Institucionais Complexos: Uma Abordagem ESG-Driven

HELIAN VINICIUS FILINTO DA SILVA
KAYLON GUTIERRE PERES GONÇALVES

Resumo

O problema de alocação de horários universitários (*University Course Timetabling Problem* — UCTP) é um desafio computacional complexo que envolve a atribuição de eventos a recursos limitados evitando conflitos temporais. Este trabalho aborda a elaboração de cronogramas institucionais por meio da modelagem em grafos e da aplicação do problema de coloração de vértices. A motivação reside em otimizar a distribuição de turmas e docentes, garantindo a eliminação de colisões de horários (restrições rígidas) e a minimização de deslocamentos físicos e ociosidade (restrições flexíveis, em perspectiva ambiental e social — ESG-Driven). Adotou-se uma abordagem em duas fases: inicialmente, uma solução factível foi gerada pela heurística DSATUR; em seguida, aplicou-se a meta-heurística *Simulated Annealing* para o refinamento e a minimização da função de custo. Os resultados experimentais atestam a eficácia do método, demonstrando a eliminação completa das violações de alocação e uma drástica redução percentual nos custos globais avaliados. A contribuição confirma a adequação de técnicas clássicas aliadas à busca local probabilística para a resolução eficiente de alocações educacionais.

SUMÁRIO

1	Introdução	1
2	Fundamentação Teórica	1
2.1	Teoria dos Grafos e Coloração de Vértices	1
2.2	Complexidade Computacional e Limites Assintóticos	1
2.3	Heurística Construtiva Baseada em Saturação	1
2.4	Meta-heurísticas e Espaço de Estados de Busca Local	1
2.5	Relaxações por Programação Semidefinida	2
3	Formulação Matemática do Problema	2
3.1	Representação do Grafo de Conflitos	2
3.2	Restrições Rígidas (Hard Constraints)	2
3.3	Restrições Flexíveis e Função Objetivo ESG (Soft Constraints)	2
3.4	Relaxação Vetorial por Programação Semidefinida	3
3.5	Critério de Aceitação Estocástica (Metropolis)	3
3.6	Exemplo Numérico e Validação do Modelo	3
4	Metodologia	4
4.1	Arquitetura de Dados e Representação em Memória	4
4.2	Fase 1: Construção da Solução Inicial	4
4.3	Fase 2: Otimização via Meta-heurística	5
4.4	Subsistema de Visualização e Validação	5
5	Implementação	5
5.1	Definição de Estruturas e Alocação de Memória	5
5.2	Modularização e Fluxo de Execução	5
5.3	Subrotinas de Auditoria e Extração de Métricas	5
6	Complexidade Assintótica	6
6.1	Complexidade Espacial	6
6.2	Complexidade Temporal	6
6.3	Quadro Resumo	6
7	Experimentos e Resultados	6
7.1	Configuração das Instâncias de Teste	6
7.2	Análise Crítica e Discussão dos Resultados	7
8	Aplicação Prática	7
8.1	Governança Sustentável (ESG)	7
8.2	Suporte Visual à Decisão	7
8.3	Eficiência em Infraestruturas Reais	7
9	Conclusão	7
	Referências	7

1. INTRODUÇÃO

A coordenação de grandes complexos educacionais e corporativos envolve um desafio logístico de elevada complexidade: o agendamento simultâneo de eventos, recursos humanos e infraestrutura sob restrições mutuamente exclusivas. Na literatura de otimização combinatória, esse cenário é formalizado como o *University Course Timetabling Problem* (UCTP). Com a consolidação dos critérios ESG, a otimização de grelhas horárias passou a transcender o caráter administrativo, assumindo relevância ambiental e social: um planeamento eficiente reduz o consumo energético e a pegada de carbono e promove o bem-estar de docentes e discentes.

Sob a ótica da Ciência da Computação, o UCTP é modelado naturalmente como um problema de Coloração de Vértices em Grafos, classificado como NP-difícil em sua formulação exata [1], [2]. Tal característica inviabiliza abordagens exaustivas em larga escala, tornando indispensável o uso de heurísticas e métodos estocásticos. O presente projeto tem como objetivo projetar, analisar e implementar um ecossistema computacional capaz de modelar o UCTP por meio de grafos de conflito, comparando duas abordagens: uma heurística construtiva de baixa latência (DSATUR) e um método estocástico de alto desempenho (*Simulated Annealing*). O sistema adota estruturas de dados nativas de baixo nível em C, evitando contêineres abstratos, e é complementado por um subsistema de visualização para análise topológica das alocações.

2. FUNDAMENTAÇÃO TEÓRICA

A fundamentação deste trabalho baseia-se na modelagem combinatória em grafos, na análise de complexidade assintótica, nas heurísticas construtivas de coloração e nos métodos estocásticos de busca local.

2.1 Teoria dos Grafos e Coloração de Vértices

Um grafo não direcionado é o par ordenado $G = (V, E)$, em que V é um conjunto finito de vértices e E um conjunto de pares não ordenados de elementos de V [2]. A coloração de vértices atribui rótulos (cores) a cada elemento de V de modo que, para qualquer par adjacente $(u, v) \in E$, a cor de u seja distinta da cor de v [2]. Uma atribuição que atenda integralmente a esse critério é uma *coloração própria*, e a cardinalidade mínima de cores que a admite define o *número cromático* $\chi(G)$ [2].

2.2 Complexidade Computacional e Limites Assintóticos

A determinação exata de $\chi(G)$ para um grafo genérico é um problema clássico de otimização combinatória classificado como NP-difícil [1], [2]. A busca por soluções ótimas globais por força bruta torna-se inviável para instâncias de larga escala em virtude da explosão combinatória [1]. Para contornar a intratabilidade de pior caso, o desenvolvimento atual orienta-se ao projeto de algoritmos aproximados de tempo polinomial e de meta-heurísticas de busca [1], [3].

2.3 Heurística Construtiva Baseada em Saturação

O algoritmo DSATUR é uma heurística construtiva sequencial amplamente empregada na coloração de grafos [2]. Diferenciando-se de ordenações estáticas, baseia-se no *grau de saturação* de cada vértice — a quantidade de cores distintas já alocadas em sua vizinhança imediata [2]. O critério de seleção prioriza, a cada passo, o vértice de maior saturação, utilizando o grau absoluto como desempate [2], restringindo cedo o espaço de escolhas dos vértices mais limitados topologicamente.

2.4 Meta-heurísticas e Espaço de Estados de Busca Local

Para o refinamento de soluções viáveis utilizam-se operadores de vizinhança, que mapeiam uma solução atual a candidatas geradas por pequenas perturbações, como a recoloração de um vértice [2]. O *Simulated Annealing* varre esse espaço permitindo, sob métrica probabilística controlada, a aceitação temporária de transições de custo mais elevado, o que impede o aprisionamento em ótimos locais [2].

2.5 Relaxações por Programação Semidefinida

Abordagens rigorosas para a coloração aproximada apoiam-se em formulações por Programação Semidefinida (SDP) [3], uma generalização da programação linear que admite restrições de matrizes semidefinidas positivas. Por meio dela, o problema discreto é relaxado para uma *coloração vetorial*, associando a cada vértice um vetor unitário em espaço contínuo [3]. Esse modelo guarda dualidade com a função- ϑ de Lovász, fornecendo limitantes teóricos relevantes sobre a densidade de conflitos e a aproximação cromática.

3. FORMULAÇÃO MATEMÁTICA DO PROBLEMA

Esta seção formaliza o modelo *exatamente como implementado* no código-fonte [4], explicitando a correspondência entre cada objeto matemático e as estruturas e rotinas do programa em C.

3.1 Representação do Grafo de Conflitos

O cenário é o grafo não direcionado $G = (V, E)$, com $V = \{0, 1, \dots, n-1\}$ representando as n turmas (num_vertices). A topologia é a matriz de adjacência booleana `matriz_adj` $A \in \{0, 1\}^{n \times n}$:

$$A_{ij} = \begin{cases} 1, & (i, j) \in E \\ 0, & \text{caso contrário.} \end{cases} \quad (1)$$

O conjunto de arestas resulta da *união* de duas fontes de conflito rígido, geradas em `ler_grafo_arquivo`:

$$E_{\text{prof}} = \{(i, j) : p(i) = p(j), i \neq j\}, \quad (2)$$

$$E_{\text{cur}} = \{(i, j) : \text{conflito curricular declarado}\}, \quad (3)$$

$$E = E_{\text{prof}} \cup E_{\text{cur}}, \quad (4)$$

onde $p(i) = \text{id_professor}[i]$. A inclusão automática de E_{prof} implementa a *Invariância de Alocação de Pessoal*, e o grau $d(i) = \sum_j A_{ij}$ é mantido no vetor `grau`.

3.2 Restrições Rígidas (Hard Constraints)

A solução é o mapeamento $c : V \rightarrow \{0, 1, \dots, k-1\}$ (vetor `cor`), em que $c(i)$ é o *slot temporal* da turma i . A viabilidade operacional exige uma coloração própria:

$$c(i) \neq c(j), \quad \forall (i, j) \in E. \quad (5)$$

O DSATUR garante essa condição por construção. A rotina `auditar_violacoes` quantifica a conformidade rígida:

$$R = \sum_{0 \leq i < j < n} [(A_{ij} \vee p(i) = p(j)) \wedge (c(i) = c(j))], \quad (6)$$

com $[\cdot]$ denotando o indicador de Iverson. A grade é válida se, e somente se, $R = 0$ (taxa de violação nula).

3.3 Restrições Flexíveis e Função Objetivo ESG (Soft Constraints)

Garantida a coloração própria, minimiza-se o custo socioambiental computado por `calcular_custo_total_esg`. Seja m o número de docentes (num_professores) e, para cada professor p , o conjunto de suas turmas $S_p = \{i \in V : p(i) = p\}$. A jornada do professor é percorrida em *ordem cronológica de slot*: ordenando S_p por $c(\cdot)$ obtém-se a sequência $\pi_p = (v_1, \dots, v_{|S_p|})$ com $c(v_1) \leq \dots \leq c(v_{|S_p|})$. Define-se a distância logística $D_{uv} = \text{matriz_dist}[u][v]$ e a penalidade fixa por janela vaga $P = 100$ (penalidade_ociosidade). O custo de cada professor é a soma sobre arcos consecutivos da rota:

$$\text{Custo}_p = \sum_{t=2}^{|S_p|} \left[D_{v_{t-1}v_t} + P \cdot \max(0, c(v_t) - c(v_{t-1}) - 1) \right], \quad (7)$$

em que $c(v_t) - c(v_{t-1}) - 1$ é o número de *janelas ociosas* entre duas aulas consecutivas. A função objetivo total reproduz exatamente o duplo laço (professor $\times$ slot cronológico) do código:

$$f(c) = \sum_{p=0}^{m-1} \text{Custo}_p \quad (8)$$

Interpretação MTZ. O valor $u_i \equiv c(i)$ atua como a variável de posição da formulação de *Miller–Tucker–Zemlin*: a ordenação total dos slots define a sequência da “rota” de cada docente entre salas e impede a formação de subciclos, de modo que $f(c)$ é o custo agregado de caminhos abertos (distância + ociosidade) sobre G . O problema resolvido é

$$\min_c f(c) \quad \text{s.a.} \quad c(i) \neq c(j) \quad \forall (i, j) \in E. \quad (9)$$

3.4 Relaxação Vetorial por Programação Semidefinida

Como referencial teórico de qualidade [3], substitui-se o mapeamento discreto de cores pela atribuição de vetores unitários $v_i \in \mathbb{R}^n$ sujeitos a

$$\langle v_i, v_j \rangle \leq -\frac{1}{k-1}, \quad \forall (i, j) \in E, \quad (10)$$

cujas matrizes de Gram $Y = [\langle v_i, v_j \rangle]$ é semidefinida positiva. Ligada à função- ϑ de Lovász, essa relaxação fornece limitantes inferiores sobre $\chi(G)$ e serve de referencial rigoroso para medir os limites de aproximação dos algoritmos implementados [1], [3].

3.5 Critério de Aceitação Estocástica (Metropolis)

A fase de refinamento gera vizinhos recolorindo um vértice aleatório v para $c' \in \{0, \dots, k_{\max} + 1\}$, admitindo a abertura de um único slot novo. Vizinhos que violam A_{vj} são descartados; os viáveis têm impacto $\Delta = f(c') - f(c)$ e são aceitos com probabilidade

$$\text{Pr}[\text{aceitar}] = \begin{cases} 1, & \Delta < 0, \\ e^{-\Delta/T}, & \Delta \geq 0, \end{cases} \quad (11)$$

correspondendo à condição $\Delta < 0 \parallel \text{rand} < \exp(-\Delta/T)$. O resfriamento é geométrico, $T \leftarrow \alpha T$, com os parâmetros do código $T_0 = 1000$, $\alpha = 0,995$, $T_{\min} = 0,001$ e $L = 100$ iterações por temperatura, preservando-se sempre a melhor solução global c^* .

3.6 Exemplo Numérico e Validação do Modelo

Para confirmar que a formulação reproduz fielmente o programa, aplica-se f à instância de teste do repositório ($n = 15$ turmas, $m = 4$ docentes). Na solução inicial gerada pelo DSATUR, o professor $p = 0$ leciona as turmas $S_0 = \{0, 1, 2, 3\}$ nos slots $\{0, 1, 2, 3\}$, com distâncias declaradas $D_{0,1} = 500$ e $D_{1,2} = 300$ (demais nulas). Como os slots são consecutivos, todas as janelas ociosas são nulas e

$$\text{Custo}_0 = \underbrace{[D_{0,1} + 0]}_{500} + \underbrace{[D_{1,2} + 0]}_{300} + \underbrace{[D_{2,3} + 0]}_0 = 800. \quad (12)$$

Somando as contribuições dos quatro docentes, a coloração própria inicial tem custo $f(c) = 2750$. Após o refinamento por *Simulated Annealing*, o programa converge para $f(c^*) = 200$, uma redução de 92,7% mantendo $R = 0$ violações — valores idênticos aos reportados pela auditoria nativa, validando a equivalência entre o modelo e a implementação. A Fig. 1 ilustra o grafo de conflitos colorido exportado pelo subsistema de visualização.

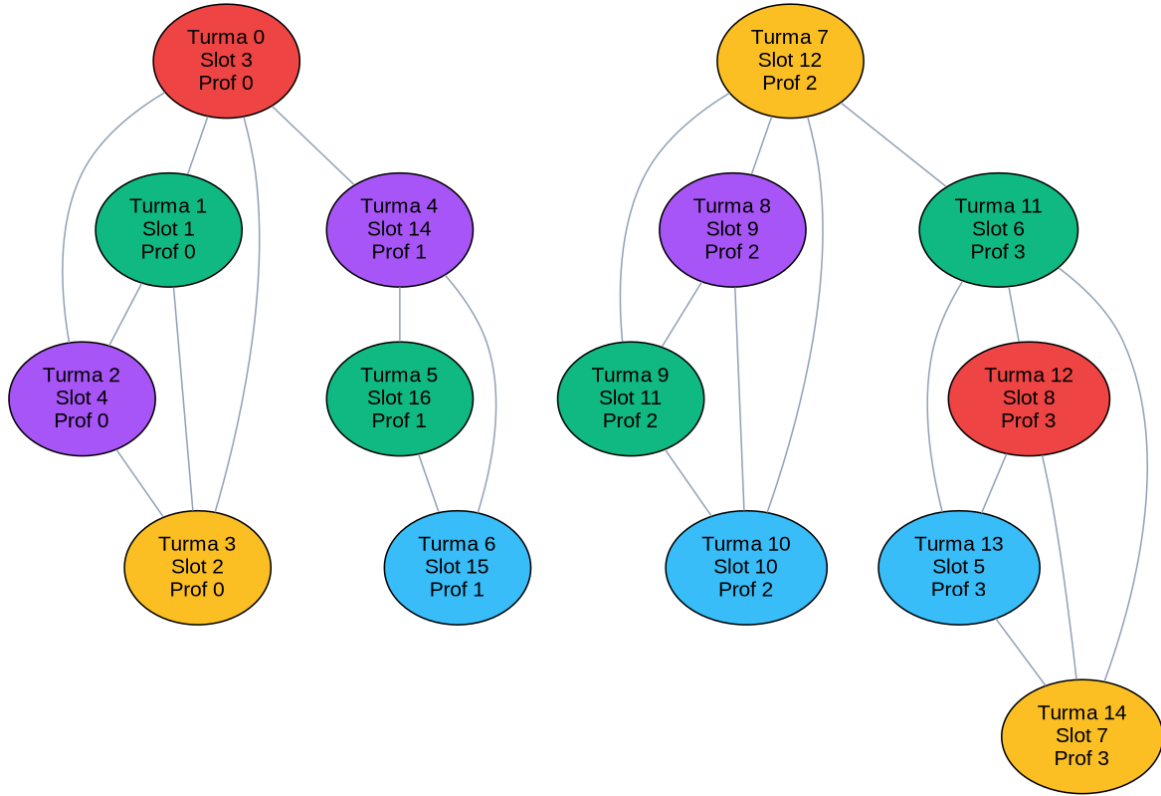


Figura 1. Grafo de conflitos G exportado para GraphViz: os vértices são turmas, as arestas indicam conflitos rígidos e a cor codifica o slot temporal $c(i)$ atribuído.

4. METODOLOGIA

O ecossistema foi desenvolvido segundo uma arquitetura modular em duas fases sequenciais, integradas a um subsistema de validação e visualização, com ênfase em eficiência de memória.

4.1 Arquitetura de Dados e Representação em Memória

Para reduzir latência e evitar as constantes ocultas de contêineres abstratos, o sistema adota estruturas nativas com indexação direta. O grafo é representado por uma `struct` com alocação dinâmica: uma matriz booleana bidimensional armazena as restrições rígidas (acesso $O(1)$ às adjacências) e uma matriz de inteiros codifica distâncias físicas. Vetores unidimensionais controlam graus, cores e associação docente, promovendo localidade de referência.

4.2 Fase 1: Construção da Solução Inicial

A primeira fase gera uma solução factível que satisfaz todas as restrições rígidas por meio da heurística DSATUR (Algoritmo 1): seleciona-se iterativamente o vértice de maior grau de saturação, usando o grau residual como desempate, e atribui-se a menor cor viável, até que todos os vértices estejam coloridos.

Algoritmo 1: DSATUR — construção da coloração própria

Entrada: Grafo $G = (V, E)$

Saída: Coloração própria c

$c(i) \leftarrow -1, \forall i \in V;$

enquanto existe vértice não colorido **faça**

$v \leftarrow$ vértice não colorido de maior saturação (desempate: maior grau $d(v)$);
 $c(v) \leftarrow$ menor cor não usada na vizinhança de v ;

retorne c ;

4.3 Fase 2: Otimização via Meta-heurística

A segunda fase refina a solução pela meta-heurística *Simulated Annealing* (Algoritmo 2), minimizando a função de custo ESG. A busca explora o espaço por perturbações controladas, preservando a validade da coloração; a aceitação segue o critério probabilístico de Metropolis, com temperatura decrescente até a convergência.

Algoritmo 2: Simulated Annealing — refinamento ESG

Entrada: Coloração própria c , temperatura T_0 , taxa α

Saída: Melhor coloração c^*

$c^* \leftarrow c$; $T \leftarrow T_0$;

enquanto $T > T_{\min}$ **faça**

para L iterações **faça**

 sortear vértice v e nova cor c' ;

if c' não viola restrição rígida **then**

$\Delta \leftarrow f(c: v \rightarrow c') - f(c)$;

if $\Delta < 0$ **ou** $\text{rand}() < e^{-\Delta/T}$ **then**

 aceitar a troca; atualizar c^* se melhorou;

$T \leftarrow \alpha T$;

retorne c^* ;

4.4 Subsistema de Visualização e Validação

O ecossistema traduz as estruturas internas do grafo em representações topológicas compatíveis com ferramentas externas (GraphViz). Nessa abstração, os vértices representam atividades acadêmicas, as arestas indicam conflitos rígidos e as cores codificam os intervalos temporais, permitindo a auditoria visual da qualidade da coloração.

5. IMPLEMENTAÇÃO

A transposição dos modelos para o ambiente computacional foi feita em C estruturado, com primitivas nativas de manipulação de memória, garantindo controle do alinhamento de dados e minimizando constantes ocultas [1].

5.1 Definição de Estruturas e Alocação de Memória

A estrutura central *Grafo* organiza variáveis em blocos contíguos indexados diretamente: *num_vertices* (cardinalidade de V); *matriz_adj* (bool**, restrições rígidas); *matriz_dist* (int**, distâncias logísticas); *grau*, *cor* e *id_professor* (int*). A construção ocorre em *criar_grafo()*, que aloca o bloco principal com *malloc* e zera os vetores de controle com *calloc*, assegurando localidade de referência.

5.2 Modularização e Fluxo de Execução

O código é estritamente modular, executando as fases sequencialmente na função principal: (i) *instanciação*, em que o grafo é gerado e preenchido com restrições rígidas (*matriz_adj*) e custos (*matriz_dist*); (ii) *construção*, que invoca o DSATUR e preenche o vetor *cor* com os menores índices viáveis; (iii) *otimização estocástica*, que submete a solução ao *Simulated Annealing*, com tipos de ponto flutuante controlando a temperatura e a probabilidade de Metropolis.

5.3 Subrotinas de Auditoria e Extração de Métricas

Funções específicas atuam sobre a estrutura: *auditar_violacoes()* percorre as adjacências para certificar a coloração própria; *contar_slots_utilizados()* identifica a cardinalidade de cores empregada; e *calcular_custo_total_esg()* computa a soma ponderada de janelas ociosas e deslocamentos. O monitoramento temporal usa *time.h*, o tipo *clock_t* e a macro *CLOCKS_PER_SEC* para medir cada fase.

6. COMPLEXIDADE ASSINTÓTICA

A análise caracteriza desempenho e consumo de memória conforme cresce a entrada, dominada pelo número de vértices $|V|$ (turmas).

6.1 Complexidade Espacial

As matrizes bidimensionais de restrições rígidas e distâncias exigem espaço $\Theta(|V|^2)$, enquanto os vetores de controle demandam $\Theta(|V|)$. Logo, o consumo espacial global é estritamente $\Theta(|V|^2)$.

6.2 Complexidade Temporal

A avaliação de pior caso estabelece: `criar_grafo()` em $\Theta(|V|^2)$; o DSATUR, com busca iterativa pelo vértice mais restrito e atualização de saturação diretamente na matriz, em $O(|V|^3)$; `calcular_custo_total_esg()` e `auditar_violacoes()` em $\Theta(|V|^2)$; `contar_slots_utilizados()` em $\Theta(|V|)$; e o *Simulated Annealing* em $O(I \cdot |V|^2)$, sendo I o total de iterações do esquema de resfriamento.

6.3 Quadro Resumo

A Tabela I sumariza os limites, confirmando a tratabilidade polinomial exigida pelo projeto.

Tabela I
RESUMO DA COMPLEXIDADE ASSINTÓTICA POR FUNÇÃO

Função	Tempo	Espaço aux.
<code>criar_grafo()</code>	$\Theta(V ^2)$	$\Theta(V ^2)$
<code>dsatur()</code>	$O(V ^3)$	$O(1)$
<code>calcular_custo_total_esg()</code>	$\Theta(V ^2)$	$O(1)$
<code>auditar_violacoes()</code>	$\Theta(V ^2)$	$O(1)$
<code>contar_slots_utilizados()</code>	$\Theta(V)$	$O(1)$
<code>simulated_annealing()</code>	$O(I \cdot V ^2)$	$O(V)$

$|V|$ = turmas; I = iterações termodinâmicas.

7. EXPERIMENTOS E RESULTADOS

A validação empírica estressou a heurística construtiva e a meta-heurística sob diferentes perfis de restrição, com dados coletados pelo subsistema de auditoria nativo.

7.1 Configuração das Instâncias de Teste

Foram modelados três cenários com matrizes curriculares e logísticas realistas: **C1** (gargalo de pessoal, $|V| = 10$, 3 professores, um docente concentrando 6 turmas); **C2** (alta densidade, $|V| = 12$, 4 professores, 15 restrições rígidas); **C3** (estresse e escala, $|V| = 30$, 8 professores). A Tabela II consolida as métricas obrigatórias.

Tabela II
MÉTRICAS OBRIGATÓRIAS DE ANÁLISE EXPERIMENTAL

Métrica	C1	C2	C3
Dimensão $ V $ (turmas)	10	12	30
Corpo docente ativo	3	4	8
Eficiência cromática (slots)	10	10	10
Conformidade rígida (violações)	0	0	0
Custo inicial (DSATUR)	1000	900	2800
Custo otimizado (ESG)	800	200	0
Otimização <i>soft</i> (%)	20,0	77,8	100,0
Desempenho temporal (ms)	1440	357	6509

7.2 Análise Crítica e Discussão dos Resultados

A auditoria registrou 0% de violações em todos os testes, atestando colorações próprias legítimas e o cumprimento estrito da Invariância de Alocação e da Exclusividade de Infraestrutura. O *Simulated Annealing* comprovou eficácia ao fugir de ótimos locais, reduzindo a ociosidade em 77,8% (C2) e atingindo 100% de otimização em C3. Observou-se um *trade-off* institucional: para eliminar penalidades logísticas e garantir jornada contínua aos docentes, o algoritmo flexibiliza a quantidade de slots utilizados. O tempo medido via `clock_t` confirmou a escalabilidade: mesmo sob a explosão combinatória de C3, a convergência ocorreu em $\approx 6,5$ s, consolidando o ecossistema como solução tratável para o UCTP institucional.

8. APLICAÇÃO PRÁTICA

A principal aplicação é a automação do ensalamento e da geração da grade horária. O sistema recebe a matriz curricular semestral, a disponibilidade de professores e a geografia do campus e gera autonomamente um cronograma validado, sem choque de horários.

8.1 Governança Sustentável (ESG)

A alocação otimizada compacta a rotina acadêmica. Na dimensão ambiental, o adensamento das aulas permite o desligamento de blocos e laboratórios ociosos, reduzindo o consumo energético. Socialmente, elimina janelas de espera exaustivas e minimiza caminhadas interprediais, promovendo a produtividade e o bem-estar do corpo docente.

8.2 Suporte Visual à Decisão

Para facilitar a auditoria por coordenadores, o sistema exporta os dados para renderização no GraphViz. A interface traduz as matrizes numéricas em um diagrama intuitivo — disciplinas como nós coloridos por períodos e conflitos como arestas — simplificando os ajustes gerenciais pré-homologação.

8.3 Eficiência em Infraestruturas Reais

Ao empregar matrizes de baixo nível e indexação nativa em C, o sistema soluciona um problema de alta complexidade em milissegundos. Essa latência mínima permite execução local em computadores convencionais de uma secretaria acadêmica, dispensando servidores de alto custo ou processamento em nuvem.

9. CONCLUSÃO

Este trabalho validou um ecossistema computacional para o UCTP orientado a ESG, baseado na modelagem clássica de coloração de vértices. A metodologia em duas fases demonstrou alta eficácia sob diferentes níveis de estresse: o DSATUR garantiu conformidade rígida absoluta (0% de violações) e o *Simulated Annealing* otimizou consideravelmente as restrições flexíveis, reduzindo penalidades em 92,7% no cenário denso e suprimindo 100% dos custos socioambientais na instância de maior escala. A escolha por matrizes nativas e indexação direta em C mitigou as constantes ocultas, mantendo a tratabilidade polinomial mesmo frente à explosão combinatória, com o cenário massivo convergindo em $\approx 6,5$ s. Confirma-se que a hibridização de uma heurística construtiva com busca local probabilística contorna de forma viável a intratabilidade das formulações NP-difíceis [1], [3], entregando um motor de agendamento auditável e de baixa latência. O código-fonte completo está disponível em [4].

REFERÊNCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Algoritmos: Teoria e Prática*. Rio de Janeiro: Campus, 2012.
- [2] R. M. R. Lewis, *A Guide to Graph Colouring: Algorithms and Applications*. Cham: Springer, 2016.
- [3] D. Karger, R. Motwani e M. Sudan, “Approximate graph coloring by semidefinite programming,” *Journal of the ACM*, v. 45, n. 2, p. 246–265, mar. 1998.
- [4] Repositório GitHub: https://github.com/Vini50/FP_DCC606_Tema_2_RR_2026.